

Classi

Un'applicazione Java è composta da una o più **classi**, ciascuna con un nome che per convenzione deve sempre iniziare con una lettera maiuscola.

Una classe viene salvata in un file con estensione *.java*, che deve avere lo stesso nome della classe; questo perché Java utilizza proprio i nomi delle classi per cercare il codice ed eseguirlo.

Ad esempio, la classe *Persona* sarà salvata in un file *Persona.java*.

Tra tutte le classi, una si differenzia dalle altre per la presenza del metodo **main**, di fondamentale importanza, in quanto è proprio con questo metodo che parte l'esecuzione dell'applicazione.

```
public class NomeClasse{  
    public static void main(String[] args){  
        //istruzioni  
        ...  
    }  
}
```

Il metodo **main** viene dichiarato come:

- **public**: il metodo è pubblico ed è visibile a tutto il codice, anche all'esterno della classe;
- **static**: il metodo appartiene alla classe stessa e non ad una sua istanza specifica. Questo significa che il metodo può essere richiamato direttamente, senza creare un oggetto della classe;
- **void**: il metodo non restituisce nessun valore.

Dopo il nome del metodo, tra parentesi tonde, vengono indicati gli eventuali parametri, salvati in un vettore di stringhe chiamato *args*.

Attributi

All'interno di una classe possiamo dichiarare delle variabili che le appartengono; queste prendono il nome di **attributi**.

È buona prassi dichiarare tutti gli attributi di una classe con un livello di visibilità privato, utilizzando la parola chiave **private**. Questo perché è opportuno che dall'esterno non sia possibile accedere e modificare direttamente i valori degli attributi.

Gli attributi dipendono da cosa la classe deve rappresentare: per esempio, se vogliamo rappresentare una persona, possibili attributi possono essere il nome, il cognome, l'altezza, l'età, ...

All'interno della classe, possiamo accedere ad un attributo utilizzando la sintassi con il punto (.).

Metodi

I metodi rappresentano tutte le azioni che un oggetto di quella determinata classe è in grado di eseguire. I metodi che si vogliono rendere utilizzabili al di fuori della classe devono essere dichiarati come **public**, mentre quelli che implementano in un qualche modo una logica interna che non si vuole rendere accessibile all'esterno è bene che vengano dichiarati come **private**.

Costruttori

Sono metodi particolari che vengono utilizzati per istanziare (creare) un oggetto della classe.

Caratteristiche dei metodi costruttori:

1. devono essere **pubblici**, proprio perché devono essere accessibili all'esterno della classe per poter creare un oggetto di tale classe;
2. non viene indicato il tipo di ritorno, nemmeno **void**;
3. devono avere lo **stesso nome della classe**.

È possibile prevedere più costruttori, che devono avere parametri diversi.

Setter

Sono metodi particolari che ci consentono di andare a modificare (settare) il valore degli attributi della classe. Sono da prevedere in quei contesti nei quali occorre poter modificare dall'esterno il valore di qualche attributo: ricordiamo che gli attributi sono privati, quindi non possiamo accedervi direttamente.

Si crea un metodo setter separato per ogni attributo della classe (solo quelli che si prevede di poter modificare dall'esterno).

Caratteristiche dei metodi setter:

1. devono essere **pubblici**;
2. non prevedono tipo di ritorno, quindi sono **void**;
3. prevedono un parametro dello stesso tipo dell'attributo al quale fanno riferimento;
4. il nome di un metodo setter sarà **sempre** formato dalla parola *set* seguita dal *nome dell'attributo* con l'iniziale maiuscola;
5. impostano il valore dell'attributo con il valore del parametro.

Getter

Sono metodi particolari che ci consentono di restituire (to get = ottenere) all'esterno della classe il valore dei suoi attributi. Anche qui ricordiamo che gli attributi sono privati.

Si crea un metodo getter separato per ogni attributo della classe.

Caratteristiche dei metodi getter:

1. devono essere **pubblici**;
2. hanno come tipo di ritorno il **tipo dell'attributo** a cui fanno riferimento;
3. non prevedono parametri;

4. il nome di un metodo getter sarà **sempre** formato dalla parola *get* seguita dal *nome dell'attributo* con l'iniziale maiuscola;
5. restituiscono (tramite **return**) il valore dell'attributo.

toString

Il metodo `toString` serve a creare una stringa con tutte le informazioni dell'oggetto (in genere nome della classe e valore dei suoi attributi) e restituirli all'esterno.

Caratteristiche:

- è **pubblico**;
- ha come tipo di ritorno sempre `String`;
- non prevede parametri.

In ogni classe, questo metodo implementa overriding; questo perché la classe base generale di Java (la classe `Object`) mette già a disposizione questo metodo, che viene quindi sovrascritto da tutte le sue sottoclassi.

this

La parola chiave **this** è una **reference variable** (variabile di referenza/riferimento) che viene utilizzata all'interno di un metodo della classe per accedere ad un elemento dell'oggetto corrente (attributi e metodi).

extends

La parola chiave **extends** serve per indicare che una classe rappresenta una sottoclasse di un'altra, che viene detta classe genitore o superclasse (o sopraclasse).

La classe derivata (la sottoclasse) eredita tutti gli attributi ed i metodi della sua superclasse, e può aggiungere attributi e/o metodi.

super

La parola chiave **super** serve a fare riferimento alla superclasse di una certa classe (sottoclasse). Il suo scopo principale è quello di eliminare ambiguità che potrebbe esserci quando una classe e la sua superclasse hanno metodi con lo stesso nome.

Viene usata generalmente in due casi principali:

1. per accedere ad attributi e metodi della superclasse;
2. per richiamare il costruttore della superclasse.

Esempio: classe *Persona.java*

```
public class Persona{
    private String nome;
    private String cognome;

    //costruttore senza parametri
    public Persona() {
        this.nome = "";
        this.cognome = "";
    }

    //costruttore con parametri
    public Persona(String nome, String cognome) {
        this.nome = nome;
        this.cognome = cognome;
    }

    //setter e getter
    public void setName(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return this.nome;
    }

    public void setCognome(String cognome) {
        this.cognome = cognome;
    }

    public String getCognome() {
        return this.cognome;
    }

    @Override
    public String toString() {
        return "Persona [nome="+nome+", cognome="+cognome+"]";
    }
}
```

Esempio: classe *Studente.java*

```
public class Studente extends Persona{
    private int matricola;

    //costruttore
    public Studente(String nome, String cognome, int matricola){
        super(nome, cognome);
        this.matricola = matricola;
    }

    //setter e getter
    public void setMatricola(int matricola){
        this.matricola = matricola;
    }

    public int getMatricola(){
        return this.matricola;
    }

    @Overriding
    public String toString(){
        return super.toString() + ", matricola=" + matricola;
    }
}
```

Osservazioni:

- `super(nome, cognome)` serve a richiamare il costruttore della classe `Persona`;
- `super.toString()` serve a richiamare il metodo `toString()` della classe `Persona`.